Java 中把一个字节编码成 16 进制字符串的原理

George <www.georgeinfo.com>

1 需求描述

需要将一个字节数组(byte[])转换成一个 16 进制的字符串(HEX String)，目的是让数据可视化（因为二进制的 byte[]数组是不可视的，除非把它转化成各类可视化的具体文件，如图片、声音、图像、字符串等）。

2 代码实现

乔治工作室
专注于互联网技术研发，让生活更精彩
www.georgeinfo.com

下面的代码，是使用比较直观的方式做的转换，并没有按照 Apache Commons Hex 工具类中的方式来，因为本文主要的目的不是为了性能或扩展性好，而是为了讲解转化原理和步骤。

在 Java 中，将一个 byte[]转化成可视化的 16 进制字符串，使用如下代码：

```
66  /**
67   * Convert byte[] to hex string.<br/>
68   * 把字节数组转化为16进制字符串， 这里我们可以将byte转换成int，<br/>
69   * 然后利用Java的内置工具方法：Integer.toHexString(int)<br/>
70   * 来将int转换成16进制字符串。
71   *
72   * @param byteArray 待被转换的字节数组
73   * @return hex string ，转换完成后的16进制字符串
74   */
75  public static String bytesToHexString(byte[] byteArray) {
76      StringBuilder stringBuilder = new StringBuilder("");
77      if (byteArray == null || byteArray.length <= 0) {
78          return null;
79      }
80      for (int i = 0; i < byteArray.length; i++) {
81          int v = byteArray[i] & 0xFF;
82          String hexValue = Integer.toHexString(v);
83          if (hexValue.length() < 2) {
84              stringBuilder.append(0);
85          }
86          //将每一个字节的16进制短字符串(两个字符表示)，拼接在一起。
87          stringBuilder.append(hexValue);
88      }
89      return stringBuilder.toString().toUpperCase();
90  }
```

www.georgehu.com

上面的代码大部分很容易看懂，但是第 81 行高亮标注的部分，为什么把一个字节(byte)变成 16 进制字符串时，需要把字节与 0xFF 进行按位与(&)运算呢？

回答这个问题之前，我们先来看一下，计算机基础知识中，是如何把二进制转化成 16 进制的。

我们知道，所有的数据都可以用二进制来表示，也就是说，计算机中所有类型的数据（图片、语音图像、可执行文件、文本文件、压缩文件等等），其实都可以用数字来表示。

在计算机中，数字是用它的二进制补码来存储和计算的。

正数的补码与它的原码相同（正数的原码、反码和补码都相同），负数的补码，是保持符号位不变，在它反码的基础上加 1，公式表示为：

负数的补码 = 该数的二进制反码 + 1

正数的符号位是 0，负数的符号位是 1。

在 Java 语言中，int 类型的数字，使用 4 个字节来存储，如果数字太小，用不了 4 个字节，则如果是正数，就在高位补 0，如果是负数，就在高位补 1。

下面，我们以数字“2”和“-2”来举例，说明计算机基础定义中，是如何规定将二进制转化为 16 进制的规则的。

int 类型的数字“2”，在计算机中，它二进制数来表示是：

00000000，00000000，00000000,00000010

int 类型的数字“-2”，它的二进制数表示是：

11111111, 11111111,11111111,11111110

(上面的二进制数字中，逗号是为了查看方便，实际的计算机存储中，只有 0 和 1，没有逗号)

计算机基础知识定义中，规定将二进制转化为 16 进制的规则是：

从低位开始，每 8 个 bit 位一组，分别转化成这组二进制数字对应的 16 进制字符，高位不足 8 位时，正数补 0，负数补 1.

按照这个规则，则十进制数字“2”的二进制数：

乔治工作室
GEORGE STUDIO
make life wonderful
专注于互联网技术研发，让生活更精彩
www.georgeinfo.com

00000000，00000000，00000000,00000010

转化成 16 进制后的字符串是：

0x00000002

十进制数字“-2”的二进制数：

11111111, 11111111,11111111,11111110

转换为 16 进制后的字符串是：

0xFFFFFFFE

有了上面的基础知识储备，我们来回过头去看一下本文的问题：“为什么把一个字节(byte)变成 16 进制字符串时，需要把字节与 0xFF 进行按位与(&)运算呢？”

```
int v = byteArray[i] & 0xFF;
```

为什么要有上面这一行代码呢？

乔治工作室
专注于互联网技术研发 | 让生活更精彩
www.georgeinfo.com

因为 Java 的工具方法：

String hexValue = Integer.toHexString(v)

它的入参只接受 int 类型的值，一个 int 是 4 个字节，而我们要转化的是一个字节 byte，如果我们不经处理，直接把 byte 转换成 int，传入 Integer.toHexString(v)这个工具方法，那么 byte 转成 int 的过程中，Java 会自动把这个 byte 的 bit 位，补位成 4 个字节（32 个 bit 位），这样，对于正数的一个 byte，它高位会补位 24 个 0，对于负数的一个 byte，它的高位会被补位 24 个 1。

这样，本来一个字节的数字“2”，它的二进制是：“00000010”，

补位后变成了“00000000000000000000000000000001”。

本来一个字节的数字“-2”，它的二进制是：“11111110”，补位后变成了“11111111111111111111111111111110”。

按照二进制转换成 16 进制的规则，它们两个转换成 16 进制后的字符串，分别是：

byte 正数 2 的十六进制数字 = 0x00000**02**

(对于一个 byte 的正数 2，其实 Integer.toHexString(2)输出的 16 进制字符串是 **2**，,因为 Integer.toHexString(int v)输出的 16 进制字符串，会忽略掉对应 16 进制数字高位的 0，所以 00000002 会被输出成字符串“2”)

和

byte 负数-2 的十六进制数字 = 0xFFFFFF**FE**

（负数 byte 转换成 int 的过程中，由于 Java 会在高位 24 位自动补 1，导致按照计算机知识中，二进制转 16 进制的规则，转换出来的 16 进制数字变大了很多，实际值已经不是原来的数字“-2”了）

这样就把一个 byte 的数据，转换成了 4 个 byte 的 16 进制数据，对于正数，数字的大小没有变化，但是对于负数，因为 byte 转 int 过程中，高位补 1 的原因，转出的 16 进制数字，已经比原来的实际数字”-2”大了很多。

我们知道，一个 byte 只用一个 byte 的 16 进制表示即可，即数字“2”的 16 进制用 0x02 表示即可，数字“-2”的 16 进制，用 0xFE 表示即可，这两个数字，为了从一个 byte 转换成一个 int，高位补位了 3 个字节，都是没用用途的，白白浪费掉 16 进制字符串中的 6 个字符。

也就是说，我们明确知道，我要转化的是一个 byte，用 2 个 16 进制字符表示即可，用不着再多加上 6 个 16 进制字符来表示，

乔治工作室
专注于互联网技术研发！让生活更精彩
www.georgeinfo.com

那样只会浪费字符，无谓地增加 16 进制字符串的长度。

所以，我们需要将一个 byte 转换成 int 传入工具方法 Integer.toHexString(v)之前，明确告诉 Java，**如果这个 byte 是一个负数，那么这个 byte 转换成 int 的过程中，高位 3 个字节的 24 个 bit 位，不用给我补位成 1**。

如何做到让 Java 把一个负数 byte 转换成 int 的过程中，高位 3 个字节，无需补位成 1 呢？

那就用到了移位运算符。

int v = byteArray[i] & 0xFF;

一个字节 byte，与数字 0xFF（它的 int 类型是 4 个字节，二进制是：11111111111111111111111111111111）做“按位与”运算，如果该字节 byte 是正数，做按位与后，这个字节 byte 的二进制数还是它自己，没有变；如果是负数 byte 字节，与 0xFF 做“按位与”运算后，它的高 24 个 bit 位都变成了 0，第 8 个 bit 位保持不变。

所以，一个 byte 与 0xFF 做“按位与”运算后，赋值给一个 int 类型，其作用是：【对于正数 byte，没有变化，直接将一个 byte 按照计算机基本规则扩展成 4 个 byte，高 24 位补 0 保持不变；对于负数 byte，使得它从一个 byte 扩展到 4 个 byte 后，高 24 位补的 1，变成 0，低 8 位保持不变，也就是，将一个 byte 跟 0xFF 做“按位与”运算，然后赋值给一个 int 变量，目的是将负数 byte 扩展成 int 后的高 24 位变成 0】

乔治工作室
专注于互联网技术研发，让生活更精彩
www.georgeinfo.com

这样，经过跟 0xFF 做“按位与”运算后，正数“2”扩展成 int 后的二进制数还是保持

“00000000000000000000000000000010”

不变，负数“-2”扩展成 int 后的二进制数：

“11111111111111111111111111111110”变成了：

“00000000000000000000000011111110”

这样的二进制数，使用 Java 的工具类“Integer.toHexString(v)”转换成 16 进制以后，就成了“0x02”和“0xFE”，如果对于负数 byte，不与 0xFF 做“按位与”运算，那么数字“-2”的 byte，转换成 int，再将 int 转换成 16 进制字符串时，就变成了“0xFFFFFFFE”，会浪费掉 6 个字符的位置。

讲了这么多，其实最简单的回答就是：之所以将一个 byte 与 0xFF 做“按位与”运算，是基于计算机基础知识中，规定的二进制转 16 进制的规则，同时也是为了符合 Java 工具方法“Integer.toHexString(v)”的规则，如果“Integer.toHexString(v)”这个工具方法，对于正数高位补的 0，也照样输出，比如“00000000000000000000000000000010”就输出成：00000002，而不是“2”，那么就没有必要将 byte 跟 0xFF 做“按位与”运算了，因为那样的话，不管你怎么想法将一个负数 byte 的高 24 位变成 0，到头来经过“Integer.toHexString(v)”工具方法输出 16 进制字符串后，还是会把补位的 0 原样转换成 16 进制字符，那么就起不到“缩短 16 进制字符串长度”的作用了。

乔治工作室
GEORGE STUDIO
专注于互联网技术研发，让生活更精彩
www.georgeinfo.com

一个 byte，可以存储的数字，是-128 到+127，将一个负数 byte 扩展成 4 个 byte 的 int 类型后，再与 0xFF 做按位与(&)运算，相当于是把转换后的这个负数 int，高 24 位全部设置成 0，然后这个本来是负数的一个 byte，就变成了正数的一个 int，但是没关系，因为即使是最小的一个负数 byte 10000000，按照上面的规则变成正数 int 后，也是 00000000000000000000000010000000，也就是十进制数字 128，而我们知道一个 byte 可以存储的数字是-128 到+127，不可能出现一个 byte 存储正数 128，所以，将一个负数 byte，按照上面的规则，转变成正数 int 后，不会出现二义性，也就是不会出现一个正常的正数 byte,跟一个按照上面规则转变后的负数 byte，他们的 int 数相同的情况，这种情况不会出现。

所以，基于计算机基础知识，同时也基于 Java 工具方法“Integer.toHexString(v)”对 16 进制字符串中，纯 0 字节的舍弃原则，就需要想方设法把负数 byte 的高 24 位变成 0，以便达到缩短 16 进制字符串的目的。

乔治工作室
专注于互联网技术研发 | 让生活更精彩
GEORGE STUDIO
www.georgeinfo.com